

Il tratto **ConcurrentCache** definisce l'interfaccia di una cache thread-safe con scadenza automatica delle voci.

Descrizione

Una cache è un archivio di dati che consente di recuperare rapidamente i dati che sono stati inseriti in precedenza. Ogni dato è identificato da una chiave univoca e ha un tempo di validità (calcolato dal momento dell'inserimento) corrispondente al valore definito con la creazione della cache. Il dato viene inserito nella cache specificando una chiave ed il relativo valore (entrambi di tipo stringa). La chiave viene in seguito utilizzata per recuperare il valore associato. Se il valore associato alla chiave è scaduto, la cache si comporta come se la chiave non esistesse.

Si provveda a definire la **struct ConcurrentCacheImpl** che implementa tale tratto, garantendo la correttezza del funzionamento in presenza di richieste concorrenti da parte di più thread e garantendo la corretta gestione delle risorse.

Requisiti

- Comportamento thread-safe di tutti i metodi
- Eliminazione automatica delle voci scadute
- Corretto rilascio delle risorse quando la cache viene distrutta

Sicurezza rispetto alla concorrenza

L'implementazione deve essere **thread-safe**. Ciò significa che più thread devono poter accedere e modificare la cache contemporaneamente senza causare corse critiche o dare origine a comportamenti non definiti.

Eliminazione automatica delle voci scadute

L'implementazione deve considerare scadute le voci che sono state inserite nella cache dopo la durata indicata in fase di costruzione della cache e, in questo caso, comportarsi come se la voce richiesta fosse assente. Le voci scadute devono essere rimosse automaticamente dalla cache, indipendentemente dal fatto che vengano richieste o meno.

Corretto rilascio delle risorse quando la cache viene distrutta

L'implementazione deve procedere al corretto rilascio delle proprie risorse quando la cache viene distrutta.

```

pub trait ConcurrentCache {
    /// Creates a new cache with the specified expiration duration.
    ///
    /// # Parameters
    ///
    /// * `d` - The duration for which cache entries remain valid. When a value
    ///   is added to the cache, it will expire after this duration has elapsed.
    ///
    /// # Returns
    ///
    /// A new instance of the implementing type.
    ///
    fn new(d: Duration) -> Self where Self: Sized;

    /// Retrieves a value from the cache if it exists and hasn't expired.
    ///
    /// # Parameters
    ///
    /// * `key` - The key to look up in the cache.
    ///
    /// # Returns
    ///
    /// * `Some(Arc<String>)` - If the key exists and the value hasn't expired.
    /// * `None` - If the key doesn't exist, or the value has expired.
    ///
    fn get(&self, key: &str) -> Option<Arc<String>>;

    /// Stores a value in the cache with an expiration time.
    ///
    /// # Parameters
    ///
    /// * `key` - The key under which to store the value.
    /// * `value` - The value to store. The cache will perform internal allocation.
    ///
    fn set(&self, key: &str, value: &str);
}

```

```

pub struct ConcurrentCacheImpl {

}

```

```

use std::sync::Arc;
use std::time::Duration;

```

```

#[cfg(test)]
mod tests {
    use super::{ConcurrentCache, ConcurrentCacheImpl};
    use std::time::{Duration, Instant};
    use std::thread;

    #[test]
    fn test_basic_storage_and_retrieval() {
        let cache = ConcurrentCacheImpl::new(Duration::from_secs(10));
    }
}

```

```

// Test storing and retrieving a value
cache.set("key1", "value1");
assert_eq!(
    cache.get("key1").as_deref().map(|s| s.as_str()),
    Some("value1")
);

// Test retrieving a non-existent key
assert_eq!(cache.get("non_existent_key"), None);

// Test overwriting a value
cache.set("key1", "new_value");
assert_eq!(
    cache.get("key1").as_deref().map(|s| s.as_str()),
    Some("new_value")
);
}

#[test]
fn test_expiration() {
    // Create a cache with a very short expiration time
    let cache = ConcurrentCacheImpl::new(Duration::from_millis(100));

    // Store a value
    cache.set("key1", "value1");

    // Verify it exists immediately
    assert_eq!(
        cache.get("key1").as_deref().map(|s| s.as_str()),
        Some("value1")
    );

    // Wait for expiration
    thread::sleep(Duration::from_millis(150));
    // Verify it has expired
    assert_eq!(cache.get("key1"), None);
}

#[test]
fn test_background_cleanup() {
    // Create a cache with a very short expiration time
    let cache = ConcurrentCacheImpl::new(Duration::from_millis(50));

    // Store multiple values
    for i in 0..5 {
        cache.set(&format!("key{}", i), &format!("value{}", i));
    }

    // Verify all values exist

```

```

for i in 0..5 {
  assert_eq!(
    cache.get(&format!("{}", i)).as_deref().map(|s| s.as_str()),
    Some(format!("{}", i).as_str())
  );
}
// Wait for background cleanup (longer than expiration time)
thread::sleep(Duration::from_millis(150));
// Verify all values have been cleaned up
for i in 0..5 {
  assert_eq!(cache.get(&format!("{}", i)), None);
}
}

```

```

#[test]
fn test_different_expiration_times() {
  let cache = ConcurrentCacheImpl::new(Duration::from_millis(200));

  // Set a value that will expire quickly
  cache.set("short_lived", "value1");

  // Wait a bit
  thread::sleep(Duration::from_millis(100));

  // Set another value
  cache.set("long_lived", "value2");

  // Wait for the first value to expire
  thread::sleep(Duration::from_millis(150));

  // The first value should be gone, the second should remain
  assert_eq!(cache.get("short_lived"), None);
  assert_eq!(
    cache.get("long_lived").as_deref().map(|s| s.as_str()),
    Some("value2")
  );
}

```

```

#[test]
fn test_concurrent_access() {
  use std::sync::Arc;

  let cache = Arc::new(ConcurrentCacheImpl::new(Duration::from_secs(5)));

  // Spawn threads to write to cache
  let mut handles = vec![];
  for i in 0..10 {
    let cache_clone = Arc::clone(&cache);
    let handle = thread::spawn(move || {

```

```

        cache_clone.set(&format!("{}", thread_key{}), i), &format!("{}", thread_value{}), i));
    });
    handles.push(handle);
}

// Wait for all threads to complete
for handle in handles {
    handle.join().unwrap();
}

// Verify all values were stored correctly
for i in 0..10 {
    assert_eq!(
        cache.get(&format!("{}", thread_key{}), i).as_deref().map(|s| s.as_str()),
        Some(format!("{}", thread_value{}), i).as_str())
    );
}
}

#[test]
fn test_memory_cleanup() {
    // This test verifies that the background thread is properly terminated
    {
        // Create a cache in a nested scope
        let _cache = ConcurrentCacheImpl::new(Duration::from_millis(10));
        // Cache will be dropped when it goes out of scope
    }

    // If the Drop implementation is working correctly, the background thread
    // should be terminated and this test will complete without hanging

    // We can't directly test for memory leaks in a unit test, but this
    // at least verifies that the background thread is properly joined
    assert!(true);
}

#[test]
fn test_fast_cleanup() {
    // This test verifies that the background thread is properly terminated as soon as possible
    let t1 = Instant::now();
    {
        let _cache = ConcurrentCacheImpl::new(Duration::from_secs(3));
        thread::sleep(Duration::from_millis(10));
    }
    let t2 = Instant::now();
    assert!(t2 - t1 < Duration::from_secs(1));
}
}

```